

BBSHRRDB

IT Automation with Python

Professional Certificate • Course 3 of 7

Lab Manual

Introduction to Git and GitHub

Weeks 1 – 5

Hands-On Labs • Exercises • Practice Problems • Solutions

Intermediate Level • ~20 Hours Total

■ How to Use This Lab Manual

This Lab Manual is your hands-on companion to **Course 3: Introduction to Git and GitHub** of the BBSHRRDB IT Automation with Python Professional Certificate. Each week includes structured lab exercises designed to reinforce lecture concepts through real-world practice on your local machine or a Linux/macOS/Windows environment.

■ Lab Manual Structure — Each weekly lab follows a consistent format:

- Lab Overview — Objectives, estimated time, and prerequisites
- Concept Review — Key terms, command syntax, and tool reference
- Guided Exercises — Step-by-step walkthroughs with expected output
- Practice Problems — Independent challenges to test your skills
- Challenge Lab — An extended real-world scenario
- Answer Key — Reference solutions with commentary

Prerequisites & Setup

Requirement	Details
Git 2.x+	git --version to verify; install from git-scm.com
GitHub Account	Free account at github.com
Terminal	Bash, Zsh, Git Bash (Windows), or WSL
VS Code	Recommended editor with GitLens extension
SSH Key	Generate with ssh-keygen; add public key to GitHub
Course 1 & 2	Basic Python and command-line familiarity

■ Pro Tip

All exercises assume a POSIX-style terminal. On Windows, use Git Bash or WSL.

Commands beginning with \$ are shell commands. Type them by hand — muscle memory matters.

Create a dedicated practice folder: `mkdir ~/git-labs && cd ~/git-labs`


```
# Set your identity (required before first commit)
$ git config --global user.name "Your Name"
$ git config --global user.email "you@example.com"

# Set default editor (VS Code shown)
$ git config --global core.editor 'code --wait'

# Set default branch name to 'main'
$ git config --global init.defaultBranch main

# View all configuration
$ git config --list

# View a specific value
$ git config user.name
Your Name

# Where is the config stored?
$ cat ~/.gitconfig
```

■ Git Configuration Levels

--system : applies to all users on this machine (/etc/gitconfig)

--global : applies to all repos for your user (~/.gitconfig)

--local : applies to this repo only (.git/config)

More specific levels override less specific ones.

Part C: Initialising a Repository

A Git repository is a directory that Git tracks. Initialise one with **git init**:

```
# Create a project directory and initialise
$ mkdir ~/git-labs/week1-project && cd ~/git-labs/week1-project
$ git init
Initialized empty Git repository in ~/git-labs/week1-project/.git/

# Inspect what git init created
$ ls -la
drwxr-xr-x .git/

$ ls .git/
HEAD branches/ config description hooks/ info/ objects/ refs/

# Check the status of your new repo
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use 'git add' to track)
```

Part D: The Three-Tree Model

Git manages files across three areas. Understanding this model is the key to understanding every Git command:

Area	Description
Working Directory	Files on your disk — where you edit code
Staging Area (Index)	A snapshot prepared for the next commit; git add moves files here
Repository (.git)	Permanent history of commits; git commit saves the staged snapshot

Lab 1.2 — Practice Problems

Problem 1 — Configuration Inspector

Write a Bash script **git_config_report.sh** that: prints your global Git identity, lists the current repo's remote URLs (if any), shows the default branch name, and displays the configured editor. Format output as a readable report.

Problem 2 — Repo Explorer

Initialise three separate repos — **project-alpha**, **project-beta**, **project-gamma**. In each, set a different local user.email. Write a script that loops over all three, prints the repo name and its local email.

Problem 3 — .git Directory Anatomy

Initialise a fresh repo. Explore the .git directory and write a short description (in a file **notes.txt** inside the repo) explaining the purpose of: HEAD, config, objects/, refs/heads/, and hooks/.

Lab 1 — Challenge: Team Setup Script

■ Challenge Lab

Write a script `setup_git_environment.sh` that automates onboarding a new developer:

1. Accepts `--name` and `--email` as arguments
2. Sets global git config `user.name` and `user.email`
3. Sets `init.defaultBranch` to 'main' and `core.editor` to 'code --wait'
4. Creates `~/.gitconfig_aliases` with useful aliases:

`git st -> git status`

`git co -> git checkout`

`git lg -> git log --oneline --graph --decorate --all`
5. Appends [include] path = `~/.gitconfig_aliases` to `~/.gitconfig`
6. Prints a confirmation summary of every change made

Answer Key — Week 1

Problem 1 — Configuration Inspector Solution

```
#!/bin/bash
# git_config_report.sh

echo "====="
echo " GIT CONFIGURATION REPORT"
echo "====="
echo "Name : $(git config --global user.name)"
echo "Email : $(git config --global user.email)"
echo "Editor : $(git config --global core.editor)"
echo "Branch : $(git config --global init.defaultBranch)"
echo ""

# Show remotes only if inside a git repo
if git rev-parse --is-inside-work-tree &>/dev/null; then
echo "Remotes:"
git remote -v
else
echo "Not inside a git repository."
fi
```

WEEK
2

Using Git Locally

Lab 2.1 — Commits, History, and Undoing Changes

■ Lab Overview

Objectives: Stage and commit files; read history with git log/diff; undo changes safely

Est. Time: 60–75 minutes

Prereqs: Lab 1 complete; Git configured with identity

Part A: Tracking Files with git add and git commit

The basic Git workflow is: edit files → stage with **git add** → save permanently with **git commit**.

```
# Create a project
$ mkdir ~/git-labs/week2-project && cd ~/git-labs/week2-project
$ git init

# Create some files
$ echo "# My Project" > README.md
$ echo "print('Hello, Git!')" > hello.py

# Check what Git sees
$ git status

On branch main
Untracked files:
(use 'git add <file>...' to include in what will be committed)
README.md
hello.py

# Stage both files
$ git add README.md hello.py

# Or stage everything:
$ git add .

# Commit with a message
$ git commit -m "Initial commit: add README and hello.py"
[main (root-commit) a1b2c3d] Initial commit: add README and hello.py
2 files changed, 2 insertions(+)
```

■ Writing Good Commit Messages

Use the imperative mood: 'Add feature' not 'Added feature'

Keep the subject line under 72 characters

Separate subject from body with a blank line (for longer messages)

Reference issue numbers: 'Fix login bug (#42)'

Bad: 'fix stuff' | Good: 'Fix null pointer in user authentication'

Part B: Viewing History

```
# Basic log
$ git log

commit a1b2c3d4e5f6... (HEAD -> main)
Author: Your Name <you@example.com>
Date: Mon Jan 15 09:00:00 2025

Initial commit: add README and hello.py

# Compact one-line log
$ git log --oneline

a1b2c3d Initial commit: add README and hello.py

# Graph view (very useful with branches)
$ git log --oneline --graph --decorate --all

# Show what changed in a specific commit
$ git show a1b2c3d

# Show unstaged changes
$ git diff

# Show staged changes (what will be committed)
$ git diff --staged

# Show difference between two commits
$ git diff HEAD~1 HEAD
```

Part C: Undoing Changes

Git provides several ways to undo work, each with different effects. Choose carefully based on whether changes are committed or not:

Command	Effect
git restore	Discard unstaged changes in working directory (destructive)
git restore --staged	Unstage a file (keeps working directory changes)
git revert	Create a new commit that undoes a previous commit (safe for shared history)
git reset --soft HEAD~1	Undo last commit; keep changes staged
git reset --mixed HEAD~1	Undo last commit; keep changes unstaged (default)
git reset --hard HEAD~1	Undo last commit; discard all changes (destructive)

```
# Undo uncommitted change to a file
$ git restore hello.py

# Unstage a file (after git add)
$ git restore --staged hello.py

# Safely undo a commit that has been pushed (creates new commit)
$ git revert HEAD

# Undo last commit locally, keep changes staged
$ git reset --soft HEAD~1
```

■ ■ Danger — git reset --hard

git reset --hard permanently discards all uncommitted changes.

Never use it after pushing — it rewrites history and breaks collaborators' repos.

For shared branches, always use git revert instead.

Part D: Ignoring Files with .gitignore

```
# Create a .gitignore in the project root
```

```
$ cat > .gitignore << 'EOF'
```

```
# Python
```

```
__pycache__/
```

```
*.pyc
```

```
*.pyo
```

```
.env
```

```
venv/
```

```
# OS files
```

```
.DS_Store
```

```
Thumbs.db
```

```
# Editor
```

```
.vscode/
```

```
*.swp
```

```
EOF
```

```
# Verify a file is ignored
```

```
$ git check-ignore -v venv/
```

```
.gitignore:5:venv/ venv/
```

```
# List all ignored files
```

```
$ git status --ignored
```

Part E: Tagging Releases

```
# Create a lightweight tag
$ git tag v1.0

# Create an annotated tag (recommended for releases)
$ git tag -a v1.0.0 -m "Release version 1.0.0"

# List all tags
$ git tag

# Show tag details
$ git show v1.0.0

# Push a tag to remote
$ git push origin v1.0.0

# Push all tags
$ git push origin --tags
```

Lab 2.2 — Practice Problems

Problem 1 — Commit History Analyser

Write a Python script **git_stats.py** that uses subprocess to run git commands and prints: total number of commits, number of unique authors, the 5 most recent commit messages, and the date of the first commit.

Problem 2 — Smart .gitignore Generator

Write a Python script **gen_gitignore.py** that accepts a language/framework as an argument (python, node, java) and writes a suitable .gitignore to the current directory. Each template should cover: compiled files, dependency directories, environment files, and OS-specific files.

Problem 3 — Safe Undo Utility

Write a Bash script **safe_undo.sh** that: (a) shows the last 5 commits with git log --oneline, (b) prompts the user to enter a commit hash to revert, (c) confirms with the user before running git revert, (d) prints success or failure with a helpful message.

Lab 2 — Challenge: Automated Commit Logger

■ Challenge Lab

Build a Python script `commit_auditor.py` that analyses a git repo and produces a report:

1. Accept a repo path as an argument (default: current directory)
2. Parse git log output to extract: hash, author, date, message
3. Group commits by author and by day-of-week
4. Identify commits with poor messages (under 10 chars or generic like 'fix', 'update')
5. Write a formatted report.txt with: total commits, commits per author, busiest commit day, list of flagged poor messages
6. Bonus: generate a simple ASCII bar chart of commits per weekday

Answer Key — Week 2

Problem 1 — Commit History Analyser Solution

```
# git_stats.py
import subprocess

def run(cmd):
    return subprocess.run(cmd, capture_output=True, text=True).stdout.strip()

total = run(['git', 'rev-list', '--count', 'HEAD'])
authors = run(['git', 'log', '--format=%ae']).splitlines()
unique = len(set(authors))
recent = run(['git', 'log', '--oneline', '-5']).splitlines()
first = run(['git', 'log', '--reverse', '--format=%ad',
            '--date=short']).splitlines()[0]

print(f'Total commits : {total}')
print(f'Unique authors : {unique}')
print(f'First commit : {first}')
print('\nRecent commits:')
for line in recent:
    print(f' {line}')
```

WEEK
3

Branching and Merging

Lab 3.1 — Branches, Merges, and Conflicts

■ Lab Overview

Objectives: Create branches; merge and rebase; resolve merge conflicts; apply best practices

Est. Time: 75–90 minutes

Prereqs: Labs 1–2 complete; comfortable with commits and log

Part A: Understanding Branches

A branch is simply a lightweight, movable pointer to a commit. Branches let you develop features, fix bugs, and experiment in isolation without affecting the main codebase.

```
# List all local branches (* marks current branch)
$ git branch
* main

# Create a new branch
$ git branch feature/user-auth

# Switch to it
$ git checkout feature/user-auth
Switched to branch 'feature/user-auth'

# Create AND switch in one step (preferred)
$ git checkout -b feature/user-auth

# Modern syntax (Git 2.23+)
$ git switch -c feature/user-auth

# List all branches (including remote-tracking)
$ git branch -a

# Delete a merged branch
$ git branch -d feature/user-auth

# Force delete (unmerged branch)
$ git branch -D feature/experiment
```

Part B: Merging Branches

Merging integrates changes from one branch into another. Git performs a fast-forward merge when possible; otherwise it creates a merge commit.

```
# Scenario: merge feature/user-auth into main
$ git checkout main

# Fast-forward merge (no diverging history)
$ git merge feature/user-auth
Updating a1b2c3d..e4f5a6b
Fast-forward
auth.py | 42 ++++++++++++++++++++++
1 file changed, 42 insertions(+)

# No-fast-forward merge (always creates a merge commit)
$ git merge --no-ff feature/user-auth -m 'Merge feature/user-auth into main'

# Squash all feature commits into one before merging
$ git merge --squash feature/user-auth
$ git commit -m 'Add user authentication feature'
```

Part C: Resolving Merge Conflicts

A conflict occurs when two branches modify the same line of the same file. Git pauses the merge and marks the conflict in the file for you to resolve manually.

```
# After a conflicting merge:
$ git merge feature/user-auth
Auto-merging auth.py
CONFLICT (content): Merge conflict in auth.py
Automatic merge failed; fix conflicts and then commit the result.

# See which files are conflicted
$ git status
both modified: auth.py

# Open auth.py – it will look like this:
<<<<<<< HEAD
def login(user, password):
    return check_db(user, password)
=====
def login(username, pwd):
    return authenticate(username, pwd)
>>>>>> feature/user-auth

# Edit the file to the desired result, then:
$ git add auth.py
$ git commit -m "Merge feature/user-auth: resolve login signature conflict"

# If you want to abort the merge entirely:
$ git merge --abort
```

■ Conflict Resolution Tips

Use a visual merge tool: git mergetool (configure with VS Code or vimdiff)

Configure VS Code as merge tool: git config --global merge.tool vscode

Always read BOTH sides carefully before choosing a resolution

After resolving, run your tests before committing the merge

Communicate with teammates to understand the intent of each change

Part D: Rebasing

Rebasing re-applies your commits on top of another branch, creating a linear history. Use it for local cleanup; avoid rebasing shared/published branches.

```
# Rebase feature branch onto latest main
$ git checkout feature/my-feature
$ git rebase main
Successfully rebased and updated refs/heads/feature/my-feature.

# Interactive rebase — squash/edit/reorder last 3 commits
$ git rebase -i HEAD~3

# An editor opens listing the commits:
# pick a1b2c3d Add login form
# pick d4e5f6a Fix typo in form
# pick b7c8d9e Add form validation
# Change 'pick' to 'squash' (s) to combine commits

# If conflicts occur during rebase:
$ git rebase --continue # after resolving
$ git rebase --abort # to cancel entirely
```

■ Golden Rule of Rebasing

Never rebase commits that have been pushed to a shared/public branch.

Rebasing rewrites commit hashes. If teammates have based work on those commits, they will have painful conflicts. Rebase only your local, unshared work.

Lab 3.2 — Practice Problems

Problem 1 — Feature Branch Workflow

Create a repo with a main branch. Create a **feature/calculator** branch and add a **calculator.py** with add, subtract, multiply, divide functions (one commit each). Return to main, create a **hotfix/readme** branch, update README.md. Merge hotfix into main, then merge feature/calculator using --no-ff. Draw (or describe) the resulting commit graph.

Problem 2 — Deliberate Conflict Resolution

Create two branches from the same base commit. In both, edit the same function in **utils.py** differently. Merge the first branch into main (succeeds). Attempt to merge the second (conflict). Resolve the conflict, keeping the better implementation. Write a note in the commit message explaining your resolution choice.

Problem 3 — Interactive Rebase Cleanup

On a feature branch, make 6 small messy commits with messages like 'wip', 'fix', 'test'. Use **git rebase -i** to: squash them into 2 logical commits, reword the commit messages to be descriptive, and verify the final log looks clean.

Lab 3 — Challenge: Branch Manager Script

■ Challenge Lab

Write a Python script `branch_manager.py` with the following commands:

`list` — show all branches with last commit date and author

`clean` — list branches merged into main older than N days (default 7)

`protect` — print a warning if you are on main and about to commit directly

`graph` — call `'git log --oneline --graph --all'` and pipe to `less`

Use subprocess throughout. Handle errors gracefully (not a git repo, etc.).

Bonus: add a `--delete` flag to `'clean'` that deletes the stale merged branches.

Answer Key — Week 3

Problem 3 — Interactive Rebase Tips

```
# After making 6 messy commits on feature/cleanup:
$ git log --oneline
f6e5d4c wip
a3b2c1d fix
9e8f7a6 test
5d4c3b2 wip again
1a2b3c4 more fix
7f6e5d4 initial feature work

# Start interactive rebase on last 6 commits
$ git rebase -i HEAD~6

# In the editor, change to:
# reword 7f6e5d4 initial feature work
# squash 1a2b3c4 more fix
# squash 5d4c3b2 wip again
# reword 9e8f7a6 test
# squash a3b2c1d fix
# squash f6e5d4c wip

# Git opens editor twice to let you write new messages
# Final result:
$ git log --oneline
b1a2c3d Add input validation and error handling
d4e5f6a Implement core calculator logic
```

WEEK
4

Working with GitHub

Lab 4.1 — Remote Repositories and GitHub

■ Lab Overview

Objectives: Create GitHub repos; push/pull/fetch; fork repos; understand remote-tracking branches

Est. Time: 60–75 minutes

Prereqs: Labs 1–3 complete; GitHub account created; SSH key configured

Part A: Setting Up SSH Authentication

SSH keys let you authenticate to GitHub without entering your password every time. This is the recommended authentication method for the command line.

```
# Generate an SSH key pair (ED25519 is modern and recommended)
$ ssh-keygen -t ed25519 -C "you@example.com"

# Accept the default path (~/.ssh/id_ed25519) and set a passphrase

# Start the SSH agent
$ eval $(ssh-agent -s)

# Add your private key to the agent
$ ssh-add ~/.ssh/id_ed25519

# Copy your PUBLIC key to clipboard
$ cat ~/.ssh/id_ed25519.pub

# Copy the output, then go to:
# GitHub → Settings → SSH and GPG keys → New SSH key → Paste

# Test the connection
$ ssh -T git@github.com

Hi username! You've successfully authenticated...
```

Part B: Creating and Cloning Repositories

```
# Option 1: Create repo on GitHub first, then clone locally
$ git clone git@github.com:username/my-repo.git
$ cd my-repo

# Option 2: Push an existing local repo to a new GitHub repo
$ cd ~/git-labs/week2-project
$ git remote add origin git@github.com:username/week2-project.git
$ git branch -M main
$ git push -u origin main

# View configured remotes
$ git remote -v
origin git@github.com:username/week2-project.git (fetch)
origin git@github.com:username/week2-project.git (push)

# Add a second remote (e.g. a team fork)
$ git remote add upstream git@github.com:org/original-repo.git
```

Part C: Pushing, Pulling, and Fetching

These three commands sync changes between your local repo and the remote:

Command	What it does
git push origin main	Upload local commits to the remote branch
git pull origin main	Fetch remote commits AND merge into current branch (fetch + merge)
git fetch origin	Download remote commits but do NOT merge — inspect before integrating
git pull --rebase	Fetch + rebase instead of merge (cleaner linear history)

```
# Push current branch to origin
$ git push origin main

# Set upstream so future pushes need only 'git push'
$ git push -u origin main

# Fetch all remote changes (safe – doesn't touch working directory)
$ git fetch origin

# See what changed on remote before merging
$ git log HEAD..origin/main --oneline

# Now merge (or rebase)
$ git merge origin/main

# Pull with rebase (recommended for a clean history)
$ git pull --rebase origin main
```

Part D: Forking Repositories

Forking creates your own copy of someone else's repo on GitHub. This is the standard workflow for contributing to open-source projects.

```
# 1. Fork on GitHub (click the Fork button on the repo page)

# 2. Clone YOUR fork locally
$ git clone git@github.com:YOUR_USERNAME/original-repo.git
$ cd original-repo

# 3. Add the original repo as 'upstream'
$ git remote add upstream git@github.com:ORIGINAL_OWNER/original-repo.git

# 4. Keep your fork in sync with upstream
$ git fetch upstream
$ git checkout main
$ git merge upstream/main
$ git push origin main

# 5. Create a feature branch for your contribution
$ git checkout -b fix/spelling-in-readme
```

Lab 4.2 — Practice Problems

Problem 1 — Remote Sync Script

Write a Python script **repo_sync.py** that: fetches all remotes, compares local and remote branch states, reports branches that are ahead, behind, or diverged from their remote counterpart, and suggests the appropriate action (push, pull, or rebase).

Problem 2 — Fork and Contribute Simulation

Fork the public repo **github.com/bbshrrdb/practice-repo** (or create one yourself to simulate). Clone your fork, add upstream, make two commits on a feature branch, push to your fork, then open a pull request on GitHub. Capture the PR URL.

Problem 3 — Multi-Remote Workflow

Set up a repo with two remotes: **origin** (your GitHub fork) and **upstream** (the original). Write a Bash script **sync_fork.sh** that: fetches upstream, rebases your main onto upstream/main, pushes the updated main to your origin fork, and prints a log of the new commits.

Lab 4 — Challenge: GitHub Repository Dashboard

■ Challenge Lab

Use the GitHub REST API (no extra libraries — just urllib or requests) to build a Python script **github_dashboard.py** that:

1. Accepts a GitHub username as an argument
2. Lists all public repos with: name, description, stars, forks, last pushed date
3. For each repo, shows the number of open issues
4. Identifies the most-starred repo and the most recently updated repo
5. Writes a formatted Markdown report: **USERNAME_github_report.md**

API base: <https://api.github.com/users/{username}/repos>

Bonus: handle pagination (GitHub returns max 30 repos per page)

Answer Key — Week 4

Problem 1 — Remote Sync Script Solution

```
# repo_sync.py
import subprocess

def run(cmd):
    return subprocess.run(cmd, capture_output=True, text=True).stdout.strip()

# Fetch all remotes first
run(['git', 'fetch', '--all'])

branches = run(['git', 'branch']).splitlines()
for branch in branches:
    branch = branch.strip().lstrip('* ')
    remote = f'origin/{branch}'

    ahead = run(['git', 'rev-list', '--count', f'{remote}..{branch}'])
    behind = run(['git', 'rev-list', '--count', f'{branch}..{remote}'])

    if ahead == '0' and behind == '0':
        status = 'UP TO DATE'
    elif ahead != '0' and behind == '0':
        status = f'AHEAD by {ahead} -> git push'
    elif ahead == '0' and behind != '0':
        status = f'BEHIND by {behind} -> git pull'
    else:
        status = f'DIVERGED (+{ahead}/-{behind}) -> git pull --rebase'

    print(f'{branch:30s} {status}')
```

WEEK
5

Collaboration Workflows

Lab 5.1 — Pull Requests, Code Review, and CI

■ Lab Overview

Objectives: Open and review PRs; configure GitHub Actions CI; set branch protection rules

Est. Time: 75–90 minutes

Prereqs: Labs 1–4 complete; GitHub repo with at least two branches

Part A: Creating a Pull Request

A Pull Request (PR) is a formal proposal to merge your branch into another. It provides a space for discussion, code review, and automated checks before integration.

```
# Typical PR workflow:

# 1. Create a feature branch from main
$ git checkout -b feature/add-logging

# 2. Make commits
$ echo "import logging" >> app.py
$ git add app.py && git commit -m "Add logging configuration to app.py"

# 3. Push branch to GitHub
$ git push -u origin feature/add-logging

# 4. GitHub CLI — open a PR from the terminal
$ gh pr create --title "Add logging configuration" \
  --body "Adds structured logging using Python logging module." \
  --base main \
  --head feature/add-logging

# Or open the PR through GitHub's web UI:
# Navigate to your repo → Compare & pull request → Fill in details → Submit
```

■ Pull Request Best Practices

Keep PRs small and focused — one feature or fix per PR

Write a clear description: what changed, why, and how to test it

Link to the relevant issue: 'Closes #42' auto-closes the issue on merge

Request reviews from relevant teammates — not the whole team

Respond to review comments promptly; mark threads as resolved when done

Don't merge your own PR without at least one review in a team setting

Part B: Code Review Best Practices

```
# View a PR's diff in the terminal
$ gh pr view 7 --web # Open in browser
$ gh pr diff 7 # Show diff in terminal

# Check out a PR locally to test it
$ gh pr checkout 7

# Add a review comment via CLI
$ gh pr review 7 --comment --body "Consider extracting this into a helper function."

# Approve and merge
$ gh pr review 7 --approve --body "LGTM! Great logging setup."
$ gh pr merge 7 --squash --delete-branch

# Or merge via the GitHub web UI:
# Squash and merge (recommended for feature branches)
# Create a merge commit (preserves branch history)
# Rebase and merge (linear history, no merge commit)
```

Part C: GitHub Actions — Continuous Integration

GitHub Actions automates your build, test, and deployment pipelines. A workflow is defined in a YAML file inside `.github/workflows/`.

```
# .github/workflows/ci.yml
name: Python CI

on:
  push:
    branches: [ main, 'feature/**' ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.11'

      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt

      - name: Run tests
        run: python -m pytest tests/ -v

      - name: Check code style
        run: pip install flake8 && flake8 . --max-line-length=88
```

Part D: Branch Protection Rules

Branch protection prevents accidental force-pushes and enforces code quality gates before code reaches main. Configure these in GitHub under **Settings** → **Branches** → **Add rule**:

Rule	Effect
Require pull request reviews	N reviewers must approve before merging
Require status checks to pass	CI must pass (e.g. all tests green) before merge
Require branches to be up to date	Branch must be current with main before merging
Restrict who can push	Only certain teams/users can push directly to main
Require signed commits	All commits must have a valid GPG or SSH signature

Part E: Issue Tracking and Linking

```
# Create an issue via GitHub CLI
$ gh issue create --title "Fix divide by zero in calculator" \
--body "calculator.py line 18 raises ZeroDivisionError" \
--label "bug"

# List open issues
$ gh issue list

# Link a commit to an issue (auto-closes issue #18 when merged to main)
$ git commit -m "Handle divide by zero in calculator.py (closes #18)"

# Reference an issue without closing it
$ git commit -m "Partial fix for calculator edge cases (refs #18)"

# Close an issue manually
$ gh issue close 18 --comment 'Fixed in PR #22'
```

Lab 5.2 — Practice Problems

Problem 1 — CI Workflow for a Python Project

Create a GitHub repo with a simple Python calculator module and a test suite. Write a GitHub Actions workflow that: runs on every push and PR to main, tests on Python 3.10 and 3.11, runs pytest, checks style with flake8. Verify the workflow passes (green checkmark) in GitHub.

Problem 2 — PR Review Simulation

Pair with a classmate (or use two GitHub accounts). One person creates a PR with intentional style issues (long lines, missing docstrings). The reviewer leaves inline review comments requesting changes. The author addresses each comment, pushes new commits, and the reviewer approves and merges.

Problem 3 — Open Source Contribution Workflow

Find any open-source Python project on GitHub that accepts contributions. Fork it, find a beginner-friendly issue (look for labels: good first issue, help wanted), make a genuine improvement (fix a typo, improve a docstring, add a missing test), and open a real pull request.

Lab 5 — Challenge: Full Team Workflow Simulation

■ Challenge Lab

Simulate a professional team workflow using two GitHub accounts (or pair up):

1. Person A creates a repo, sets up branch protection (require 1 review + CI pass)
2. Person A adds a GitHub Actions CI workflow that runs pytest
3. Person B forks the repo, creates feature/user-signup branch
4. Person B implements a user signup feature with unit tests
5. Person B opens a PR — CI runs automatically
6. Person A reviews the PR, requests 2 changes with inline comments
7. Person B addresses the review, pushes new commits to the same branch
8. Person A approves and merges with squash-and-merge
9. Both sync their local repos from the updated main
10. Capture screenshots of: the green CI check, the review thread, the merged PR

Answer Key — Week 5

Problem 1 — CI Workflow Solution

```
# .github/workflows/ci.yml
name: CI

on:
  push:
    branches: [ main ]
  pull_request:
    branches: [ main ]

jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: ['3.10', '3.11']

    steps:
      - uses: actions/checkout@v4

      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v5
        with:
          python-version: ${ matrix.python-version }

      - name: Install dependencies
        run: pip install pytest flake8

      - name: Lint with flake8
        run: flake8 . --max-line-length=88 --exclude=venv

      - name: Run tests
        run: pytest tests/ -v --tb=short
```

■ Weeks 1–5 Summary

Week / Topic	Core Skills Demonstrated
Week 1: Version Control Fundamentals	git install/config, git init, three-tree model, .git anatomy
Week 2: Using Git Locally	git add/commit, git log/diff, undoing changes, .gitignore, git tag
Week 3: Branching and Merging	git branch/checkout, git merge, conflict resolution, git rebase -i
Week 4: Working with GitHub	SSH keys, git clone/push/pull/fetch, forking, remote-tracking branches
Week 5: Collaboration Workflows	Pull requests, code review, GitHub Actions CI, branch protection, issue tracking

■ Course 3 Complete — What's Next?

Congratulations on completing Course 3: Introduction to Git and GitHub!

Course 4: Troubleshooting and Debugging Techniques

→ pdb, logging, profiling, root cause analysis

Course 5: Configuration Management and the Cloud

→ Puppet, Terraform, cloud automation at scale

Courses 6–7: Automating Real-World Tasks and Career Readiness

→ APIs, email, PDF processing, AI-powered job preparation tools